Reglas de estilo y protocolo de programación en Angular

- Reglas de estilo y protocolo de programación en Angular
  - Reglas de estilo
    - .html
      - Clases
      - Instancia de objetos
      - Espaciado
    - .less
      - Espaciado
    - component.ts
      - Espaciado
  - Protocolo de programación
    - Configuración básica
    - .html
    - .less
      - Cuando usar `position: absolute`
      - Cuando usar `display: flex`
      - Como dar márgenes o espacios
      - Cuando usar `float: left` o `float: right`
    - component.ts
      - Efectos
        - Entrada suave
        - Entrada pop
        - Entrada desde abajo
        - Entrada desde arriba
        - Entrada desde la izquierda
        - Entrada desde la derecha
    - service.ts
    - Aspectos varios
      - Generación de objetos (componentes)
        - Objeto particular
          - Objeto particular simple
          - Objeto particular complejo
        - Objeto general
    - Protocolo de objetos y elementos
      - Elementos
        - Enviar
        - Cabeceras
          - Atrás
          - Título
          - Validar, guardar ...
          - boton de crear (+), boton de menu (3puntos)
      - Objetos
        - Emergentes
        - Alerta

Este documento detalla las pautas a seguir a la hora de desarrollar en Angular. Es muy importante seguir este documento al pie de la letra porque los proyectos que llevamos a cabo suelen realizarse entre varios programadores.

Reglas de estilo

Todo el código, sea el archivo que sea, debe indentarse con 3 espacios **¡OJO!** esto no es un tabulador de tres espacios, sino tres espacios simples, por ello debemos configurar nuestro editor para que al pulsar tabulador, introduzca 3 espacios. Esta configuración en Sublime Text es la siguiente: `tab_size: 3` y `translate_tabs_to_spaces: true`

.html

Clases

Si queremos nombrar una clase con varias palabras, deberemos hacerlo separando las palabras con un guión. Por ejemplo: `capa-deshabilitadora`, `contenido-boton`.

Dentro de un mismo div, como mucho puede haber otro sin sangrar, lo voy a mostrar con un ejemplo:

Esto **si** esta permitido porque dentro del div solo hay un elemento:

```
<div class="icono"><i class="fas fa-comment"></i></div>
```

Esto **no** esta permitido porque dentro del div hay dos elementos más:

```
<div class="icono"><i class="fas fa-comment"><b>Hola</b></i></div>
```

Por lo tanto debería ser así:

```
<div class="icono">
    <i class="fas fa-comment">
        <b>Hola</b>
    </i>
</div>
```

Instancia de objetos

Cuando instanciamos objetos, los atributos que este tenga, no debe obligarnos ha hacer scroll horizontal, por ello debemos poner los atributos uno debajo del otro de la siguiente manera, recuerda que esto solo es necesario si hay una cantidad de atributos suficiente como para que el código no sea visible sin hacer scroll.

```
<app-objeto class="objeto"
    *ngIf="objeto"
    @efecto
    [ngClass]="{'clase': condicion}"
    [ngStyle]="{'estilo': condicion}"
    [variable1]="0"
    (eventoAlgo1)="ejecutarEvento($event)"
    (eventoCerrar)="cerrar()" >
</app-pizarra>
```

El orden de los atributos es importante, observar que `class` siempre va el primero, además siempre tiene que ir en la misma linea que el objeto. Despues siguen los atributos como se muestran arriba

> Obviamente no todos los atributos son obligatorios

Espaciado

Es importante poner los espacios justos en los sitios justos. Nunca debe haber más de un espacio seguido y nunca debe haber un espacio en un sitio que no tiene por que haber espacio, aunque no afecte a la compilación.

Voy a mostrar unos ejemplos de malos espacios, espacios que faltan o sobran (marco los espacios con +):

Dos espacios que no pintan nada:

```
<div+class="boton+cerrar"++(click)="eventoCerrar.emit()"> <!-- MAL -->
<div+class="boton+cerrar"+(click)="eventoCerrar.emit()">  <!-- BIEN -->
```

Un espacio olvidado al final:

```
<div+class="boton+cerrar"+(click)="eventoCerrar.emit()"+> <!--  MAL -->
<div+class="boton+cerrar"+(click)="eventoCerrar.emit()">  <!--  BIEN -->
```

Sin espacio delante de `(click)` donde sí debería haver un espacio:

```
<div+class="boton+cerrar"(click)="eventoCerrar.emit()">  <!--  MAL -->
<div+class="boton+cerrar"+(click)="eventoCerrar.emit()"> <!-- BIEN -->
```

Espacios no necesarios dentro de `div`, si queremos añadir un espacio en blanco lo haremos con ` `:

```
<div+class="nemo">+Petición de Tutoría++</div> <!--  MAL -->
<div+class="nemo">Petición de Tutoría</div>    <!-- BIEN -->
```

Espacios que sobran dentro de `(click)`:

```
<div+class="boton+cerrar"+(click)="+eventoCerrar.emit()"> <!--  MAL -->
<div+class="boton+cerrar"+(click)="eventoCerrar.emit()">  <!--  BIEN -->
```

Lo siguiente esta prohibido, entre las expresiones condicionales tiene que haber un único espacio:

```
<span+*ngIf="obtenerAlumnoSeleccionado()!==0">   <!--  MAL -->
<span+*ngIf="obtenerAlumnoSeleccionado()+!==+0"> <!--  BIEN -->
```

.less

Nunca debe haber letras mayúsculas.

Espaciado

Después de cada sentencia hay un único espacio, por ejemplo:

```
width: 100%;
```

Debemos dejar un linea vacía antes de cada hijo, por ejemplo:

```
:host{
   display: block;
   width: 100%;
   height: 100%;
LINEA VACÍA -------------------------------
   .cabecera{
      width: 100%;
      height: 50px;
      position: relative;
   }
}
```

Entre elementos del mismo nivel, no se deja ninguna linea vacía.

component.ts

Espaciado

Al igual que en los demás ficheros, el espaciado es fundamental para la correcta comprensión del código.

A continuación voy a poner ejemplos de como debe estar espaciado el código. Prestar mucha atención que que así es exactamente como y donde habrá que poner los espacios. Estos son los espacios exactos:

Declaración de variables:

```
disponibilidad: any = [];
```

Las declaraciones de funciones:

```
comprobarValidez() {
    // CODIGO
}

mostrarOpciones($event, item, itemIndex) {
    // CODIGO
}
```

Ifs de una línea:

```
return this.fechas.modo !== 0 && this.obtenerAlumnoSeleccionado() !== undefined;
```

Ifs normales:

```
if ( !this.enviado ) {
    // CODIGO
} else {
    // CODIGO
}
```

Suscripciones:

```
this.sub.sus = this.servicio.sus.subscribe(data => {
    // CODIGO
});
```

Intervalos y Timeouts:

```
setTimeout(() => {
    // CODIGO
}, 300);

setInterval(() => {
    // CODIGO
}, 300);
```

Protocolo de programación

Configuración básica

Cuando creamos un nuevo proyecto se creará `ng new nombreproyecto --skip-tests`

Cada vez que generamos un `component`, `module`, o `service`, tenemos que añadir la opción `--skip-tests` para que no se generen archivos de test

Necesitamos configurar el proyecto para que podamos importar componentes y modulos con rutas absolutas, para ello tenemos que hacer las siguientes modificaciones en el fichero `tsconfig.json`

Reemplazamos la linea que tiene `baseUrl…` por la siguiente:

```
"baseUrl": "src"
```

A continuación de esa linea añadimos lo siguiente:

```
"paths": {
   "@app/*": ["app/*"]
}
```

Hay que sustituir el fichero styles.less por el fichero que se aporta.

.html

Es obligatorio comentar todos los divs que tenga un tamaño considerable y también hay que comentar los divs que sean difíciles de entender a simple vista.

.less

Debido a una peculiaridad en iOS, todos los `overflow: auto` irán de la siguiente manera, es la única declararción que se deja hacer en una linea:

```
overflow: auto; -webkit-overflow-scrolling: touch;
```

Hay que tener mucho cuidado a la hora de dar estilo, cada situación requiere sus reglas y hay que prevenir usar siempre las mismas reglas para todo, a continuación hay unas pautas para aprender cuando y como deberían usarse ciertas reglas.

Cuando usar `position: absolute`

Esta regla afecta a la propia capa en la que se usa. Solamente debe utilizarse `position: absolute` cuando queremos que una capa se encuentre por encima de otra o si sabemos que una capa puede llegar a estar por encima de otra.

Si una capa lleva `position: absolute`, obligatoriamente tiene que haber una capa padre con `position: relative` para indicar a que capa es relativa.

Todas las capas que llevan `position: absolute` obligatoriamente tienen que llevar al menos una coordenada horizontal `left` o `right`, y una coordenada vertical `top` o `bottom`.

Cuando usar `display: flex`

Esta regla afecta a los hijos de la capa donde se usa. Su principal uso, es para alinear los hijos que lleva dentro, para ello esta regla necesita incluir otras reglas como `justify-content`, `align-items` y `flex-direction`, entre otras.

- `justify-content`: alinea los hijos horizontalmente si `flex-direction: row` y verticalmente si `flex-direction: column`.
- `align-items`: alinea los hijos verticalmente si `flex-direction: row` y horizontalmente si `flex-direction: column`.
- `flex-direction`: indica la dirección en la que se alinean los hijos.

Esta regla también es util en diseños reposive, en el 99% de los casos debe ir acompañada de alguna reglas anteriormente descritas, asi que si vas a usar estar regla 'sola' tienes que tener muy claro por que lo haces

Como dar márgenes o espacios

Por lo general las capas hijos tienen que ocupar el 100% del espacio de la capa padres, si queremos que haya espacios, se hace con `padding` o con `margin`.

Cuando usar `float: left` o `float: right`

Cando tienes varios objetos y quieres que esten uno al lado del otro horizontalmente, debes usar `float: left` o `float: right`. También puedes conseguir el mismo efecto añadiendo estas dos reglas en cada elemento: `display: inline-block` y `vertical-align: top`.

component.ts

Es obligatorio comentar todas las funciones excepto casos muy obvios donde el nombre de la función ya la explica.

Las variables deben declararse de la siguiente manera:

- Las variables globales van siempre en mayúscula
- Las variables locales van siempre en minúsculas. Las variables que se formen por varias palabras deben estar separadas por un guión bajo, por ejemplo: `constructor_fechas`, `height_listado_fechas`, etc...
- Los eventos `@Output ()` se declaran en minúsculas. Los que se formen por varias palabras deben escribirse juntas con la primera letra de cada palabra en mayúsculas, excepto la primera, por ejemplo: `eventoCerrar`, etc...
- Las comillas que utilizamos dentro de las funciones son siempre comillas simples.
- Cuando declaramos una variable y no va ha cambiar su valor se declara con conts y no con let.

El orden de las variables debe ser el siguiente:

```
@Input()
.
.
@Output()
.
.
@ViewChild()
.
.
VARIABLES_GLOBALES (por ejemplo: BASE_APP)
.
.
variables_locales
```

Todas las variables deben declararse con el tipo para el que vayan a usarse, estos son algunos de ellos:

`array: any = [];`

`objeto: any = {};`

`numero: number;`

`texto: string;`

`boleano: boolean;`

No debemos inicializar variables en la función `ngOnInit`, todo lo que debamos inicializar lo haremos en la propia declaración. Es posible que en algún caso específico, si debamos inicializar una variable en `ngOnInit` pero esto pasará en casos muy particulares.

Todos los objetos que tengan suscripciones, deben declarar la variable `sub: any = {}` donde guardaremos la suscripción para luego en la función `ngOnDestroy` realizar una desuscripción.

> Todas, absolutamente todas las suscripciones a variables debe desuscribirse.

Todos los objetos que necesiten emitir un evento de cerrar, declararán el `Output` `eventoCerrar`:

```
@Output() eventoCerrar = new EventEmitter();
```

Si un objeto necesita acceder a las variables globales del proyecto, lo importará de la siguiente manera:

```
import { Globals } from '@app/globals';
```

Todos los objetos deben contemplar el `backbutton` (atrás) de los dispositivos Android, por lo tanto todos los objetos que necesiten emitir un evento de cerrar, tendrán que implemementar la captura y manejo del `backbutton`:

```
@HostListener('document:backbutton', ['$event'])
onBackButton() {
    this.cdRef.detectChanges();
    this.cdRef.reattach();
    this.eventoCerrar.emit();
}
```

Efectos

La mayor parte de los efectos que se utilizan, son idénticos en todos los objetos, por ello para mantener la coherencia, a continuación estan los efectos que ya estan definidos. Está prohibido usar el mismo efecto con otro nombre.

Entrada suave

```
trigger('inOutFade', [
    transition(':enter', [
        style({opacity: '0'}),
        animate('400ms', style({opacity: '1'}))
    ]),
    transition(':leave', [
        animate('400ms', style({opacity: '0'}))
    ])
])
```

Entrada pop

```
trigger('PopUp', [
    transition(':enter', [
        style({transform: 'translate(0, 0) scale(0)'}),
        animate('250ms ease', style({transform: 'translate(0, 0) scale(1)'}))
    ]),
    transition(':leave', [
        style({transform: 'translate(0, 0) scale(1)'}),
        animate('250ms ease', style({transform: 'translate(0, 0) scale(0)'}))
    ]),
])
```

Entrada desde abajo

```
trigger('inOutFromBottom', [
    transition(':enter', [
        style({transform: 'translateY(100%)'}),
        animate('400ms ease', style({transform: 'translateY(0%)'}))
    ]),
    transition(':leave', [
        animate('400ms ease', style({transform: 'translateY(100%)'}))
    ])
])
```

Entrada desde arriba

```
trigger('inOutFromTop', [
    transition(':enter', [
        style({transform: 'translateY(-100%)'}),
        animate('400ms ease', style({transform: 'translateY(0%)'}))
    ]),
    transition(':leave', [
        animate('400ms ease', style({transform: 'translateY(-100%)'}))
    ])
])
```

Entrada desde la izquierda

```
trigger('inOutFromLeft', [
    transition(':enter', [
        style({transform: 'translateX(-100%)'}),
        animate('400ms ease', style({transform: 'translateX(0%)'}))
    ]),
    transition(':leave', [
        animate('400ms ease', style({transform: 'translateX(-100%)'}))
    ])
]),
```

Entrada desde la derecha

```
trigger('inOutFromRight', [
    transition(':enter', [
        style({transform: 'translateX(-100%)'}),
        animate('400ms ease', style({transform: 'translateX(0%)'}))
    ]),
    transition(':leave', [
        animate('400ms ease', style({transform: 'translateX(-100%)'}))
    ])
]),
```

Durante el desarollo de los proyectos, siempre surgen efectos propios de ese proyecto, eso es normal, pero se exige que si se crea un efecto nuevo, ese efecto lleve el mismo nombre en todo el proyecto.

Hay libertad de crear todos los efectos que se necesiten, es importante que el nombre del efecto describa la acción del mismo.

Si fuera necesario utilizar un efecto de los ya definidos, pero con alguna pequeña modificación, el nuevo efecto resultante deberá llevar un nombre compuesto por el nombre original mas una descripción del nuevo cambio, por ejemplo, si necesitamos utilizar el efecto `inOutFromRight` pero queremos cambiar el `translate` de `%` a `vw`, el nombre del nuevo efecto resultante sería `inOutFromRightVw`.

service.ts

Todas, absolutamente todas las funciones deben llevar un comentario explicando lo que hacen, los comentarios no pueden ser más anchos que la mitad de la pantalla, para ello usaremos varias lineas si fuera necesario, además el comentario debe tener la siguiente estructura:

```
/*
 * nombreDeLaFuncion: comentario
 */
LINEA VACÍA -------------------------------
nombreDeLaFuncion() {}
```

En el fichero funciones_general.ts que se aporta, encontraras funciones generales que se suelen utilizar generalmente en los proyectos, en estas funciones se indican donde deben incluirse y su funcionalidad.

Aspectos varios

Generación de objetos (componentes)

Antes de generar un objeto (componente) es necesario pararse a pensar el destino que se le dará a dicho objeto. Se nos pueden dar varias situaciones:

Objeto particular

Este tipo de objeto se subdivide a su vez en dos:

Objeto particular simple

Este tipo de objeto no va a tener hijos que cuelguen de el, además este objeto solo se instancia desde un mismo sitio, por lo tanto no tendrá ni `module` ni `service`.

Objeto particular complejo

Este tipo de objeto puede tener hijos y además puede ser instanciado desde distintos objetos padre, para ello será obligatorio que tenga `module`. Este objeto solo tendrá `service` si lo necesita.

Objeto general

Si estamos generando un objeto que nos puede valer en cualquier parte del proyecto, debemos seguir varias pautas:

1. Generarlo en la carpeta de `objetos-comunes`.
2. Generar `module.ts`.
3. Dentro del `module.ts` es muy importante **declarar** y **exportar** el componente.

Es importante tener en cuenta que estos objetos serán globales a todo el proyecto, por lo tanto hay que diseñarlos de tal manera. Por ello, está prohibido que estos objetos tengan elementos particulares de un objeto concreto. Estos objetos serán configurables mediante `Input()` para que cada objeto que los llame, puedan configurarlos como proceda.

Protocolo de objetos y elementos

En este apartado se definiran el tipo de objetos y elemntos protocolarizados.

El fichero app.html debe tener una capa contenedora que contenga todo el bastidor. Se llamará app-bastidor.

Elementos

Enviar

Tendrá 3 estados, enviar, enviando y enviado. El enviando tiene que tener una duranción de 2 sg. color `#1ab726`

Cabeceras

Por lo general las cabeceras de los objetos tendrán un alto de 50px;

Las cabeceras se componen:

- boton atrás o Home
- Título (no obligatorio)
- validar, guardar, siguiente o boton de crear (+), boton de menu (3puntos)

Atrás

Ira siempre en el lado izquierdo del objeto en la cabecera.

```
position: absolute;
top: 0;
height: 100%;
display: flex;
justify-content: center;
align-items: center;
font-size: 1.2rem;
color: #0088ff;
font-weight: bold;
padding: 0 20px;
```

Título

```
position: relative;
font-weight: 700;
display: flex;
font-size: 1rem;
text-transform: uppercase;
justify-content: center;
align-items: center;
padding-left: 0;
letter-spacing: 1px;
top: 3%;
```

Validar, guardar ...

Ira siempre en el lado derecho del objeto en la cabecera.

El estilo sera igual que el atrás .

boton de crear (+), boton de menu (3puntos)

incluirá el estilo del validar y el siguiente:

```
font-size: 1.4rem;
-webkit-text-stroke: 2px white;
```

Objetos

Los objetos comunes estan protocolarizados y no se pueden modificar sin previo aviso.

Emergentes

Los objetos emergentes llevan una capa semitransparente y un cerrar arriba.

La capa semitransparente sera:

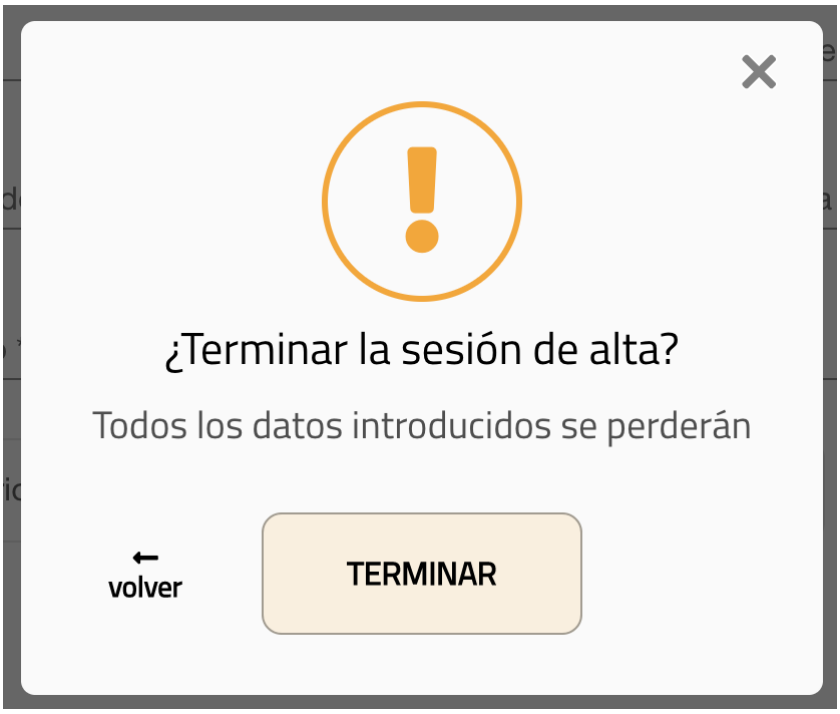
```
.capa-desabilitadora{
    background-color: rgba(0,0,0,.7);
    width: 100%;
    heigth: 100%;
}
```

El boton-cerrar:

```
.boton-cerrar{
    width: 50px;
    height: 50px;
    position: absolute;
    top: -25px;
    right: -25px;
    display: flex;
    justify-content: center;
    align-items: center;
    background: #fff;
    border-radius: 10000px;
    font-size: 1.4rem;
    border: 1px solid #ccc;
    z-index: 3;
    -webkit-text-stroke: 1px #fff;
}
```

Alerta

La importancia de este objeto reside en el boton principal tiene que estar centrado y marcado, tiene que destacar del boton secundario que tiene que ser menos importante.

Todas las alertas tienen que tener la x igual que el ejemplo abajo mostrado.

Cuando se hagan click fuera se tiene que cerrar la alerta.